Next talk

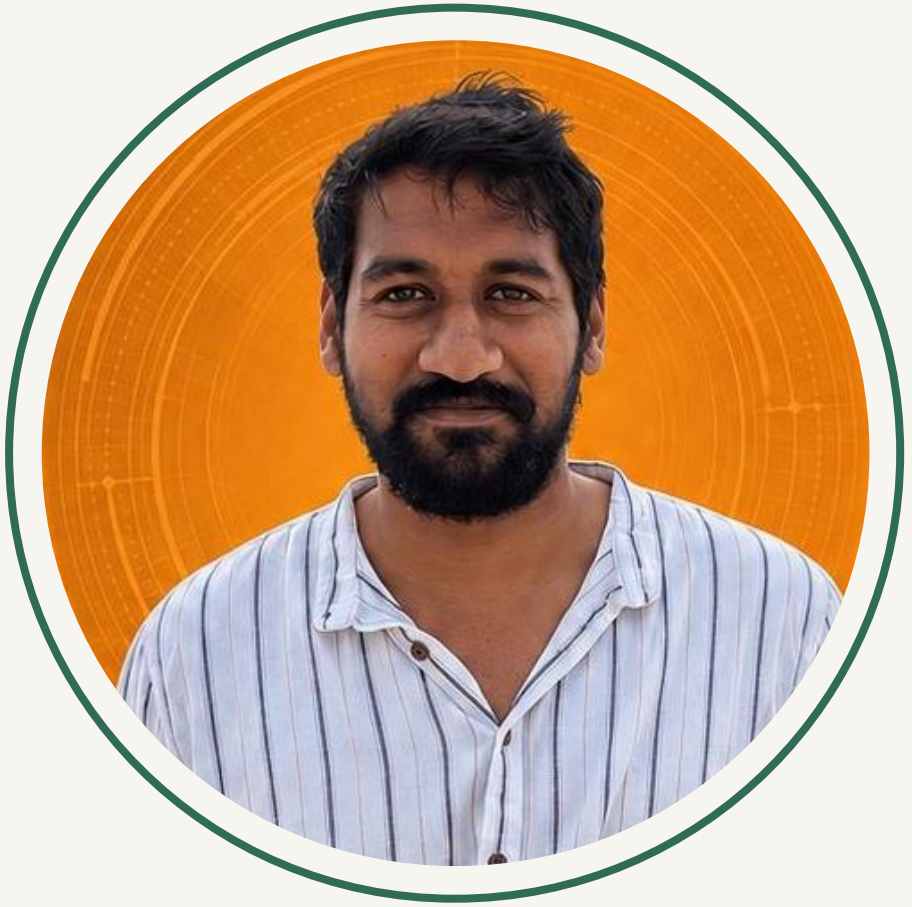
Transforming Pharma: the promise of AI in developing better medicines faster

Kas Subramanian

Thursday, Oct 29 · 9:00 PM EDT / 6:00 PM PDT



Do More Dollars Always Buy More AI Intelligence?



Avinash Reddy | PhD Student, UCF

Thursday, Oct 15 · 9pm ET / 6pm PT · Zoom Webinar

AI agents are ubiquitous



Coding

Write, debug and ship code from a prompt

`Claude Code, Cursor`



Deep research

Search the web, read sources, write cited reports

`Deep Research, Perplexity`



Computer use

Click, type and navigate apps like a person

`Operator, Claude`



Customer support

Resolve tickets and answer questions 24/7

`Intercom Fin, Sierra`



Data analysis

Query data, run analyses and build charts

`ChatGPT, Julius`



Personal assistant

Triage email, schedule meetings, book travel

`Copilot, Gemini`

What Are AI Agents?

Definitions, architecture, and capabilities

01 **What Are AI Agents?**

02 Auditing AI Agents

03 Deep Research Agents

04 Test-Time Scaling in Agents

05 Choosing Agents for Production

06 Why Scaling Fails

07 Beyond Agent Correctness

An agent is a harness plus LLM backbone

Agent = Harness + LLM

Harness

the code you write around the model

LLM backbone

weights, context, decoding

A HARNESS, IN FULL

```
def agent(question, budget):
    notes = LLM("break this into sub-questions")
    while budget.left() and not done(notes):
        query = LLM("what should I search for next?")
        notes += search(query)
    return LLM("answer using only these notes")
```

LLM() is the backbone every other line is the harness

The simplest harness is a system prompt



LLM

Answers anything, in any style

→



+ System prompt

Fixed instructions sent with every request

→



= Travel-planner agent

Same weights, new behavior

THE HARNESS IN CODE

```
SYSTEM = "You are a travel planner. Ask for dates and budget first."
def travel_planner_agent(msgs):
    reply = llm([SYSTEM] + msgs)          # "Plan me a trip to Lisbon."
    return reply.text                     # "Which dates, and what is your budget?"
```

Tool Augmented Harness

USER What will the weather be in Orlando tomorrow?

LLM



Decides to call a tool

`weather("Orlando")`

→

HARNESS



Runs the tool

`→ 84°F, 40% rain`

→

LLM



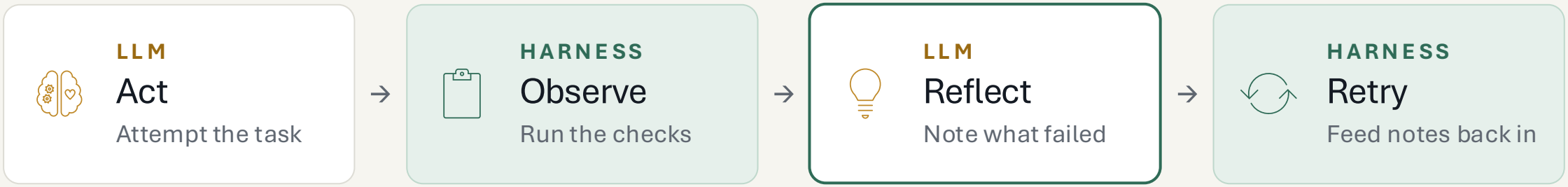
Answers from the result

Warm, with a 40% chance of rain. Pack an umbrella.

THE HARNESS IN CODE

```
def weather_agent(msgs):
    reply = llm(msgs, tools=[weather])          # weather("Orlando")
    if reply.tool_call:
        result = run(reply.tool_call)           # 84°F, 40% rain
        reply = llm(msgs + [result])
    return reply.text
```

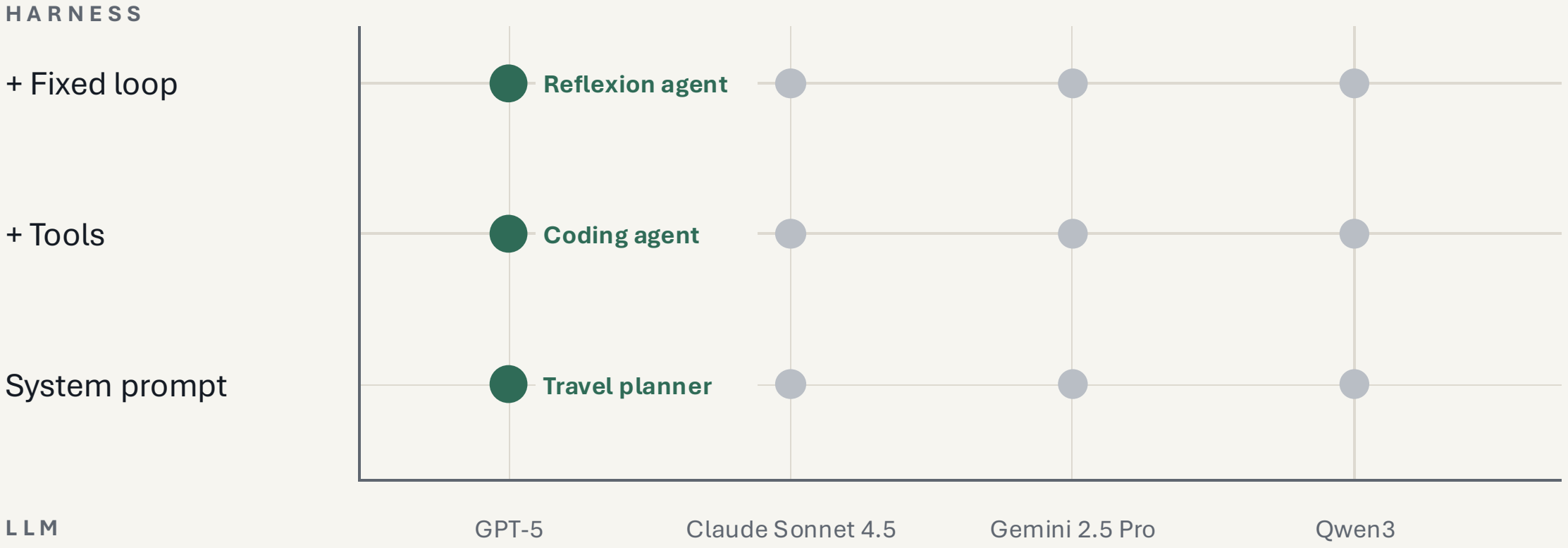
A Fixed-Loop Harness



REFLEXION, IN FULL

```
def reflexion(task, budget, notes=[]):
    while budget.left():
        attempt = LLM(task, notes)              # act
        result = run_checks(attempt)            # observe
        if result.passed: return attempt
        notes += LLM("what went wrong?")        # reflect
    return attempt
```

Same LLM, three harnesses, three kinds of agent



Each dot is an agent: one LLM inside one harness.

What does it cost to run an AI agent?

Evaluating correctness, reliability, and cost

01 What Are AI Agents?

02 Auditing AI Agents

03 Deep Research Agents

04 Test-Time Scaling in Agents

05 Choosing Agents for Production

06 Why Scaling Fails

07 Beyond Agent Correctness

Counting the inference cost

1 PROMPT TO THE LLM ~400 input tokens

```
"Forecast next quarter's sales from sales_2025.csv.
 Write Python and run it in the sandbox."
```

▼

2 LLM OUTPUT ~1,000 output tokens

```
df = pd.read_csv("sales_2025.csv")
model = fit_forecast(df, trials=200, gpu=True)
print(model.predict("Q1 2026"), model.score())
```

▼

3 SANDBOX EXECUTION GPU sandbox, ~20 min, billed per second

```
$ sandbox.run(code)
→ GPU container started, 200 trials executed
→ 4.2M  0.91
```

▼

4 LLM FINAL ANSWER ~100 output tokens

```
"Q1 2026 sales forecast: $4.2M (fit score 0.91)."
```

INFERENCE BILL

STEPS 1, 2, 4 LLM tokens	$0.02
STEP 3 Sandbox	$0.38
Total	$0.40



Tokens were **5%** of the bill

TAKEAWAY Inference cost covers every tool call and sandbox run, not just the LLM tokens.

Deep Research Agents

Multi-step reasoning, search, and synthesis

01 What Are AI Agents?

02 Auditing AI Agents

03 Deep Research Agents

04 Test-Time Scaling in Agents

05 Choosing Agents for Production

06 Why Scaling Fails

07 Beyond Agent Correctness

Deep Research Task

THE QUESTION

"Which stadium was designed by an architect who also built a train station?"

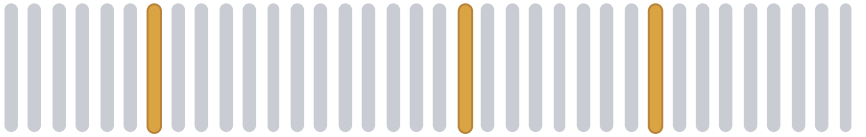
Indirect clues, many constraints

→

DEEP RESEARCH AGENT

LLM backbone
+ search · open · find

DOCUMENT CORPUS ~100K documents



gold = evidence documents

TOOLS AVAILABLE TO THE DEEP RESEARCH AGENT

SEARCH
`search(q)`
K search results



title, url, brief description

```
search("architect station stadium")
[1] doc/482 Stadium history…
[3] doc/203 Club merged in 1994…
```

OPEN
`open(url)`
the whole page

→ **full page information** many tokens

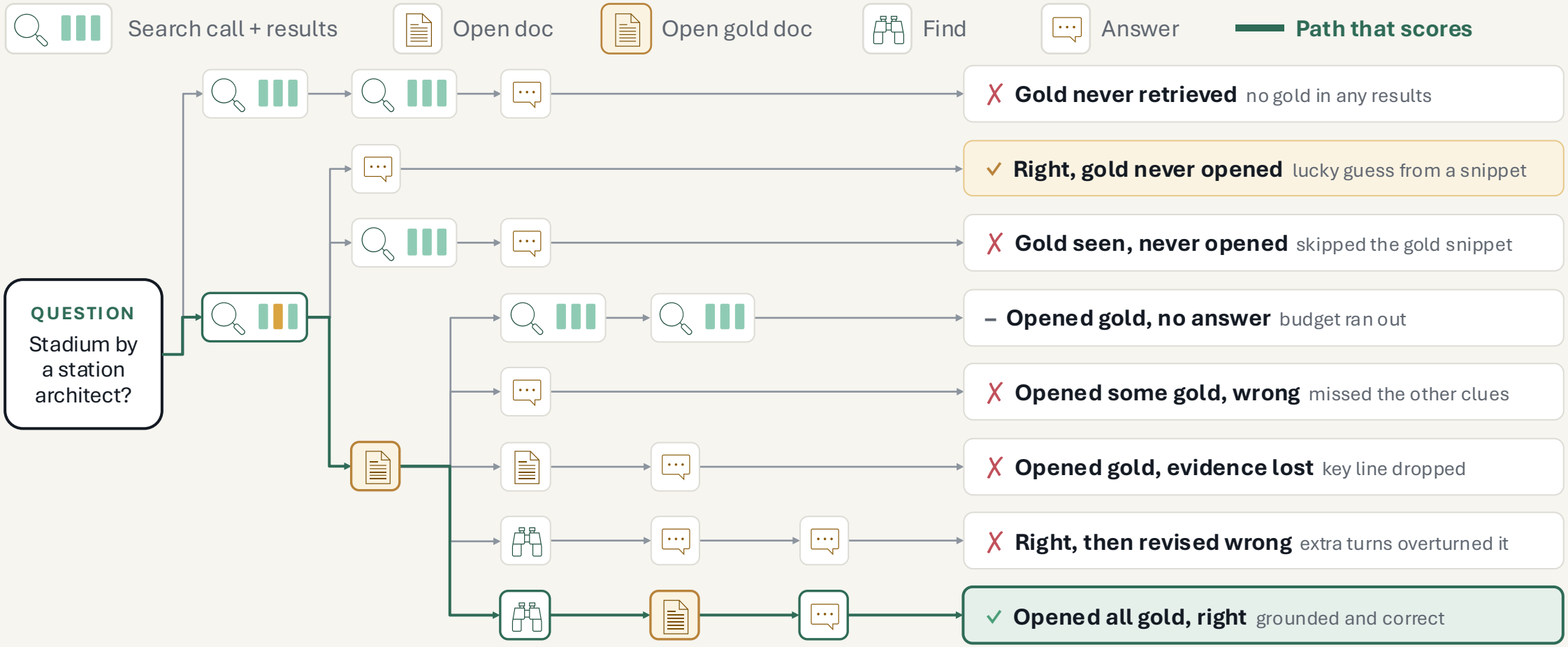
```
open("doc/203")
…renamed in 1994 after a merger.
Its stadium, opened in 1923, was…
```

FIND
`find(url, txt)`
text around a match




```
find("doc/203", "designed by")
« designed by J. Ortega »
who also built the station…
```

How a Deep Research Run Can Unfold



TAKEAWAY Gold documents must be in the agent's context for it to reach the right answer.

Scaling in Agents

Does more budget improve performance?

01 What Are AI Agents?

02 Auditing AI Agents

03 Deep Research Agents

04 Test-Time Scaling in Agents

05 Choosing Agents for Production1

06 Why Scaling Fails

07 Beyond Agent Correctness

Scaling Strategies



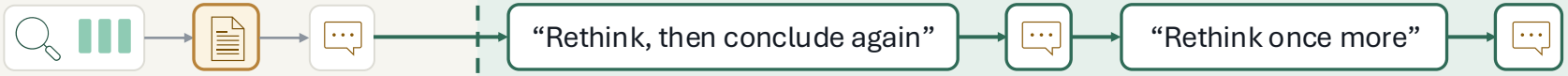
NATURAL TERMINATION | BEYOND IT = SCALING

Natural run
Agent stops when it judges it is done



Ends here: 1 search, 1 doc, 1 answer

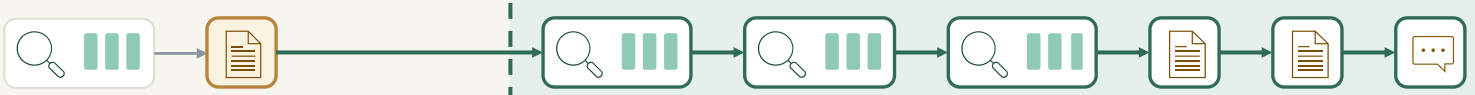
Scaling by prompt
Append a nudge after the answer



+2 reasoning passes, no new evidence

Scaling by budget
Force at least P searches and Q doc opens

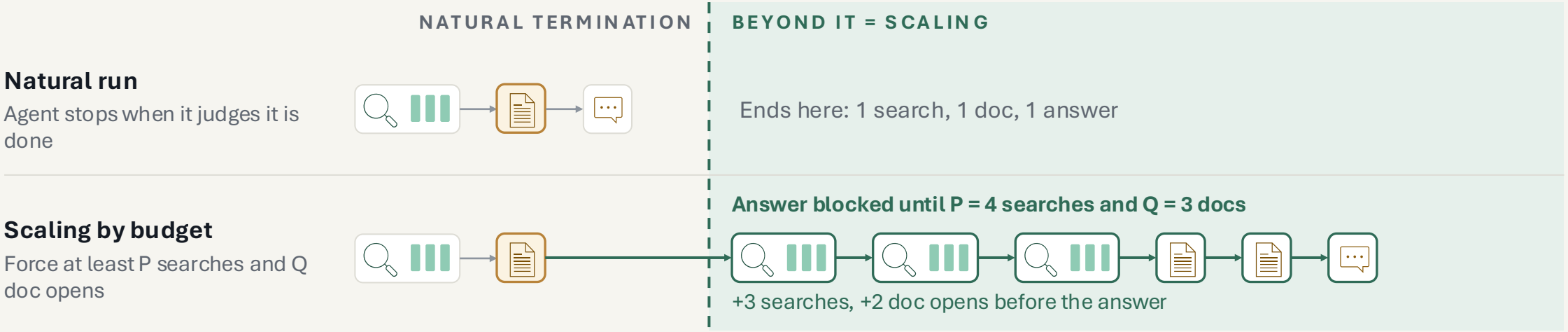
Answer blocked until P = 4 searches and Q = 3 docs



+3 searches, +2 doc opens before the answer

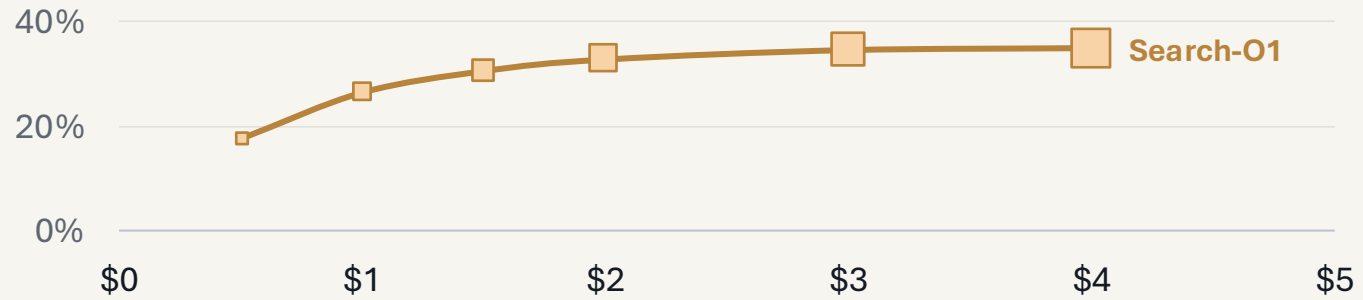
Scaling = anything that extends the trajectory past natural termination. Every added node is more inference and more cost.

Scaling the Agent improves the performance



Agent accuracy by cost per question, %

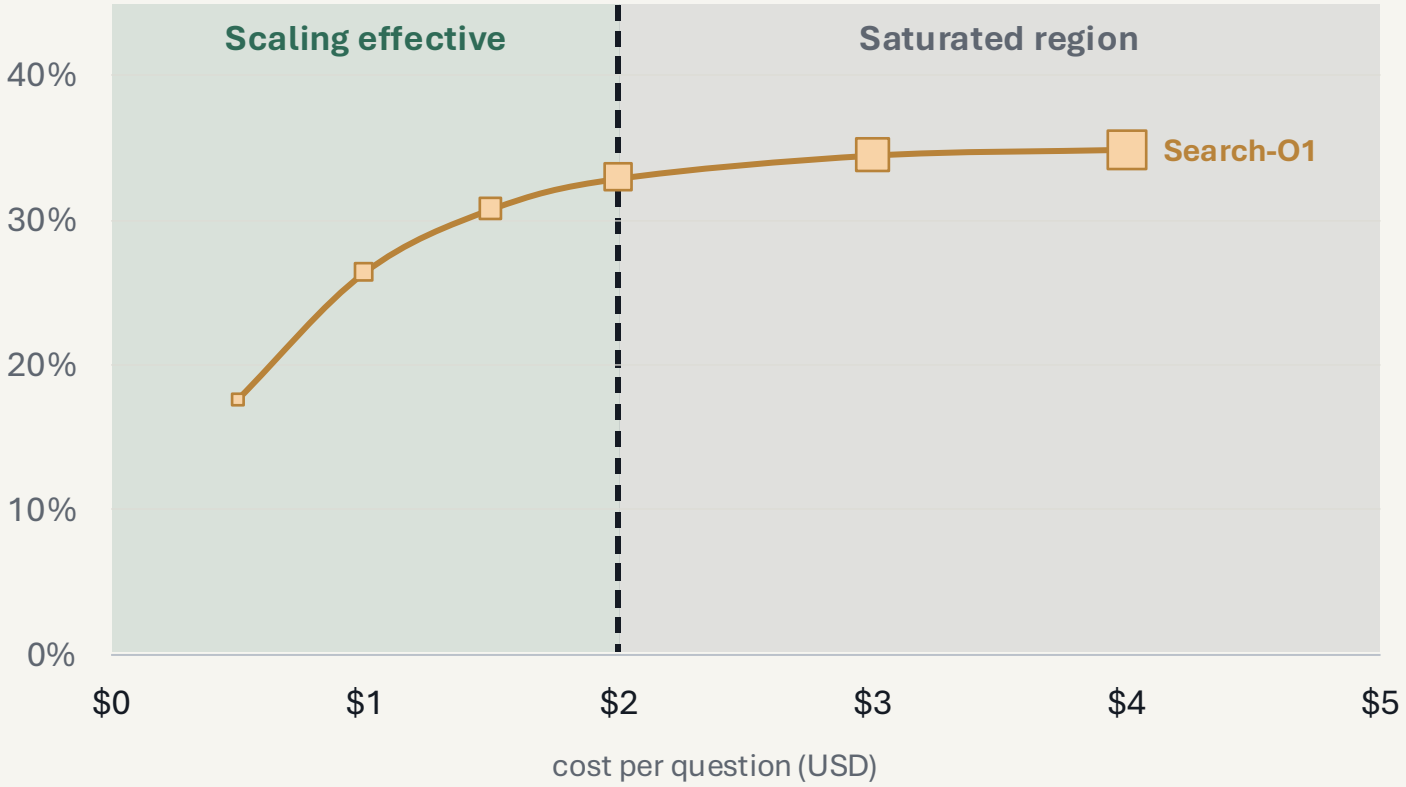
larger dot = larger budget



Scaling increases the cost per task and the accuracy

Scaling Hits a Ceiling

Search-O1 accuracy by cost per question, %
one agent; larger dot = larger budget

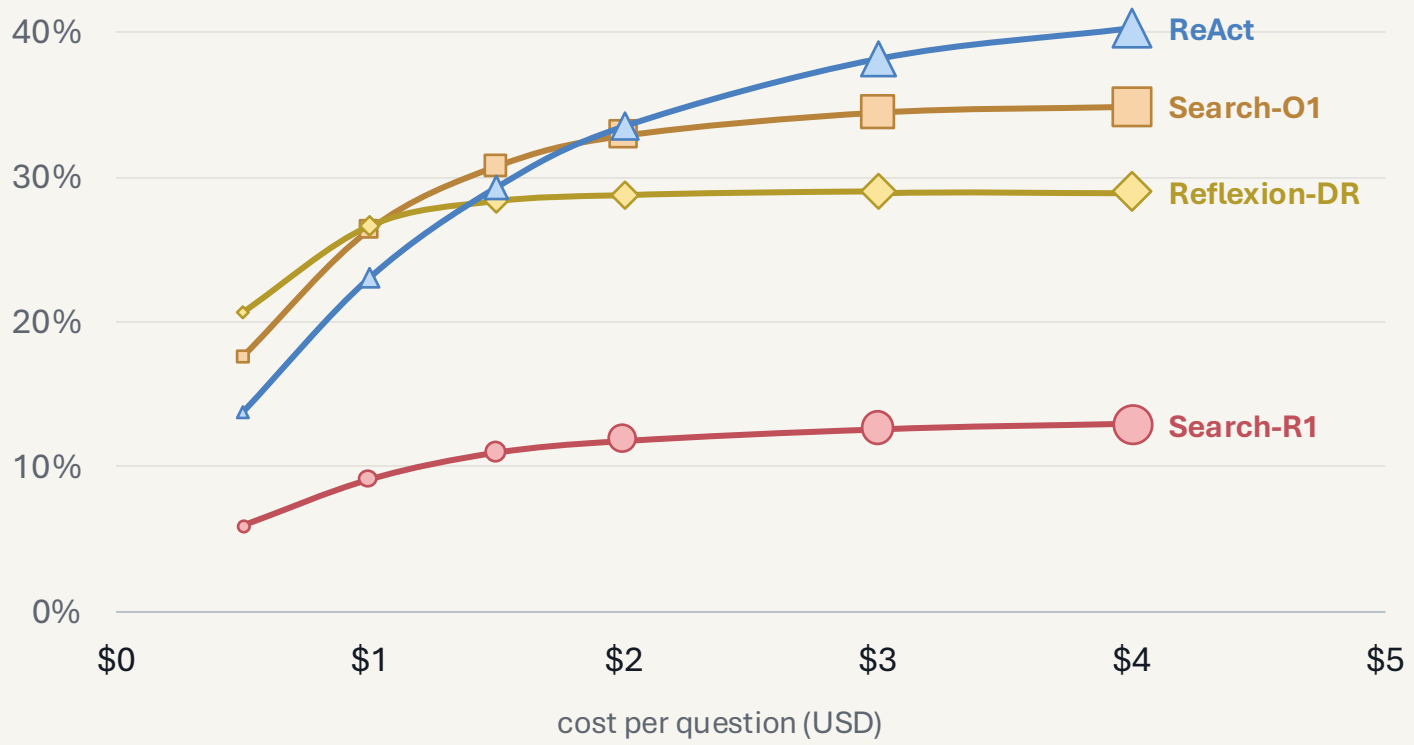


Accuracy saturates after a point

Budget is not used efficiently

Every agent saturates

Accuracy and cost per question, %
dashed line = cost frontier, most accuracy at each cost



Every agent saturates

How to Choose Agents for Scaling

Balancing accuracy, cost, and efficiency

01 What Are AI Agents?

02 Auditing AI Agents

03 Deep Research Agents

04 Test-Time Scaling in Agents

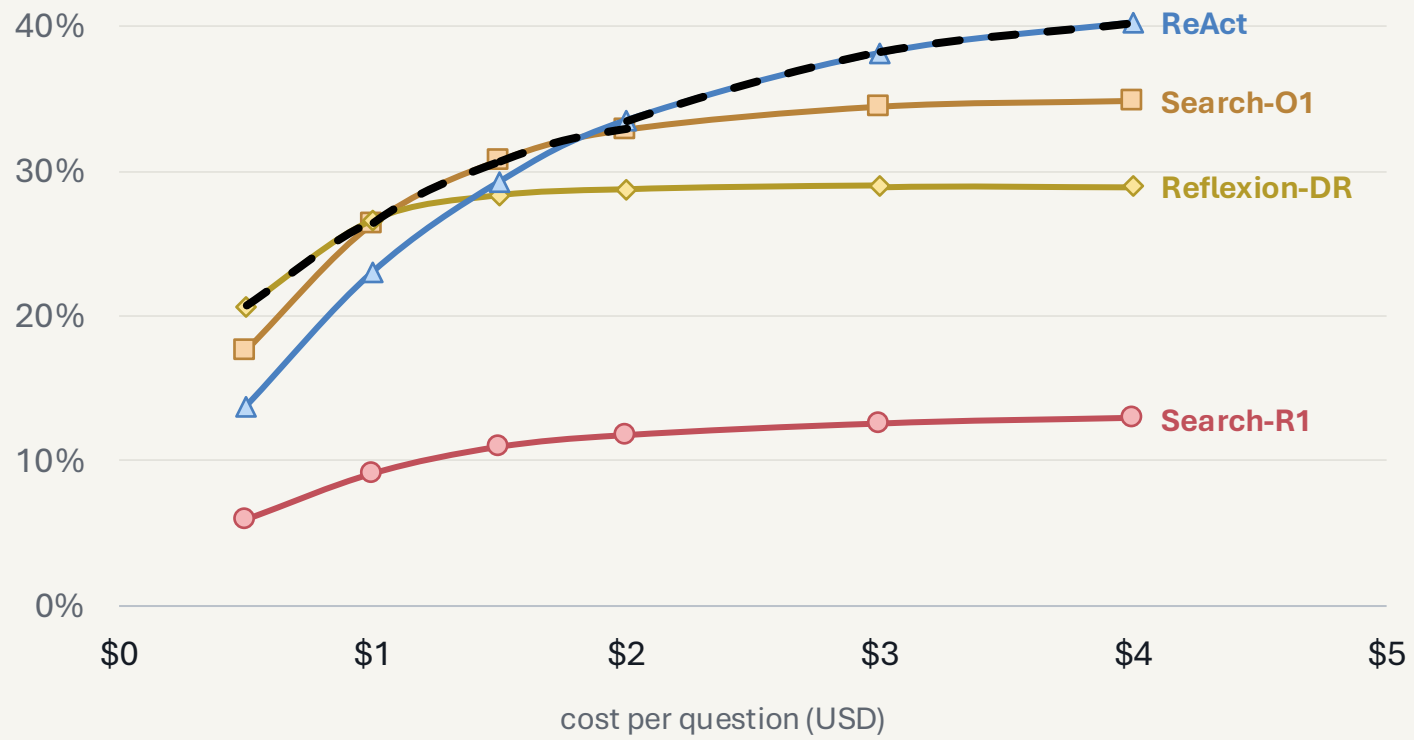
05 Choice of Agents

06 Why Scaling Fails

07 Beyond Agent Correctness

Every agent saturates, so scaling further means switching agents

Accuracy and cost per question, %
dashed line = cost frontier, most accuracy at each cost



BEST AGENT Reflexion-DR → Search-O1 → ReAct

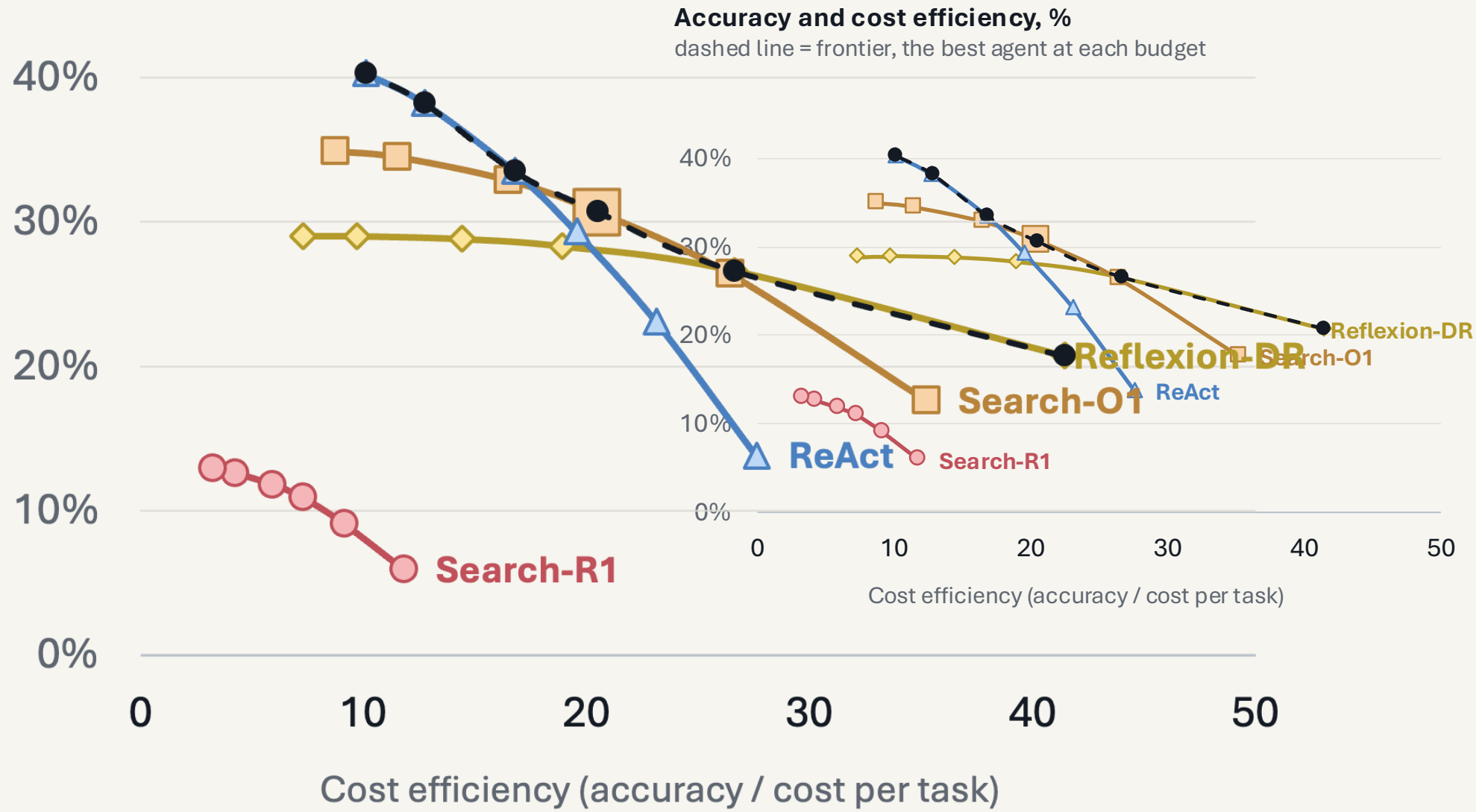
Every agent saturates

Takeaway

Switch along the frontier while you scale

Accuracy and cost efficiency, %

dashed line = frontier, the best agent at each budget



Why Scaling Fails in Agent

Diminishing returns, redundant exploration, and rising costs

01 What Are AI Agents?

02 Auditing AI Agents

03 Deep Research Agents

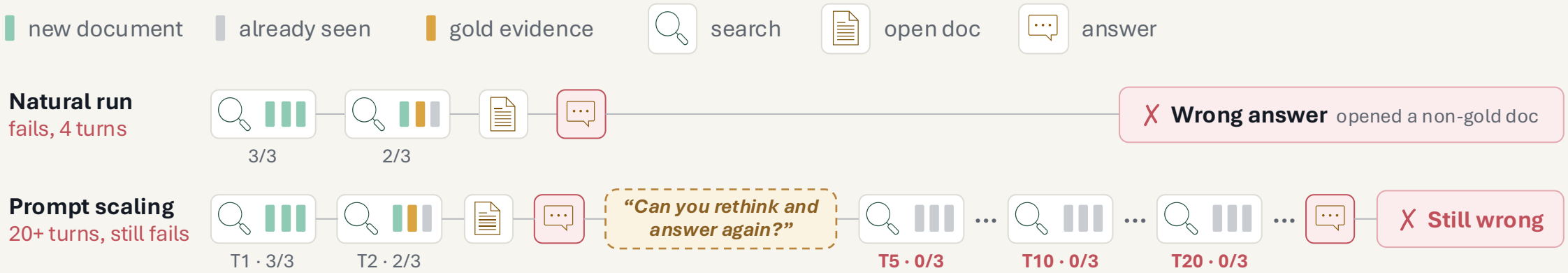
04 Test-Time Scaling in Agents

05 Choosing Agents for Production

06 Why Scaling Fails

07 Beyond Agent Correctness

Tracking Query Novelty, New Documents and Similarity



Query novelty new keywords per search

New documents per search

Semantic similarity to earlier queries

1.0

0.5

0.0

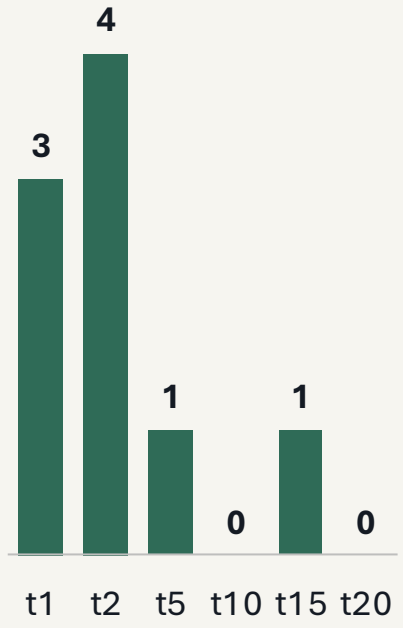
Later Searches Repeat Earlier Ones

THE AGENT'S SEARCHES

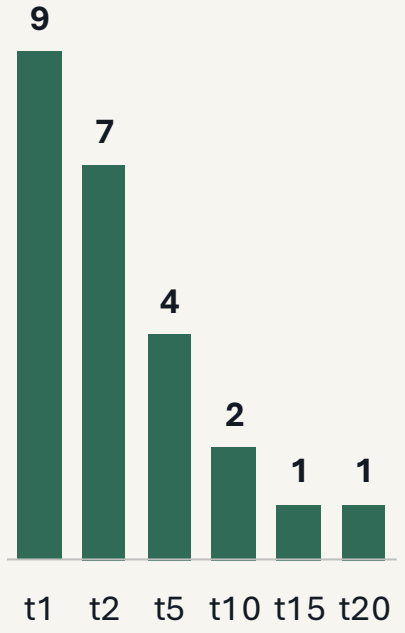
t1	stadium architect station
t2	club renamed merger 1990s
t5	stadium club history
t10	club stadium architect merger
t15	renamed club stadium designer
t20	stadium club merger renamed

Green: new directions. Gold: rewordings that mix t1 and t2.

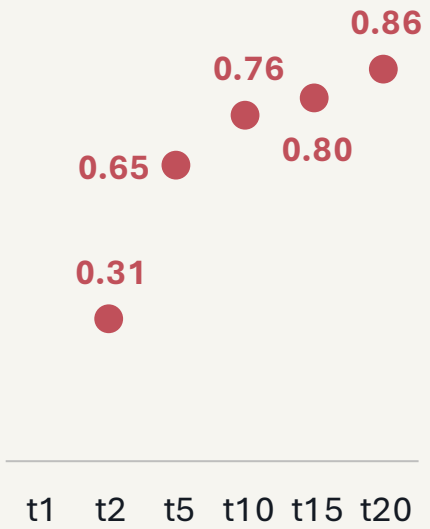
Query keyword novelty
new keywords per search



New documents
per search



Avg. semantic similarity
to earlier queries



TAKEAWAY More budget buys more searches, not new documents, so accuracy flattens.

Impression

Diverse search queries bring new documents into the agent’s context.

New evidence helps the agent reach the right answer and spend its budget efficiently.

Beyond Agent Correctness

Optimizing deployment decisions, not just agent performance

7.0 Deployment strategy preliminaries

7.1 Application-aware deployment decisions

01 What Are AI Agents?

02 Auditing AI Agents

03 Deep Research Agents

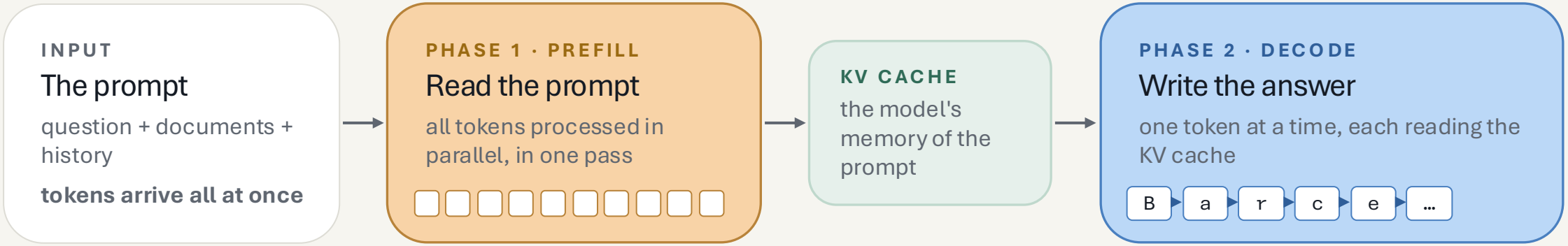
04 Test-Time Scaling in Agents

05 Choosing Agents for Production

06 Why Scaling Fails

07 Beyond Agent Correctness

Every LLM call has two phases: prefill reads, decode writes



	PREFILL	DECODE
New prompt	input tokens	output tokens, ~5× pricier
Prompt seen before	cached tokens, a fraction of the input price	never cached: every token is new

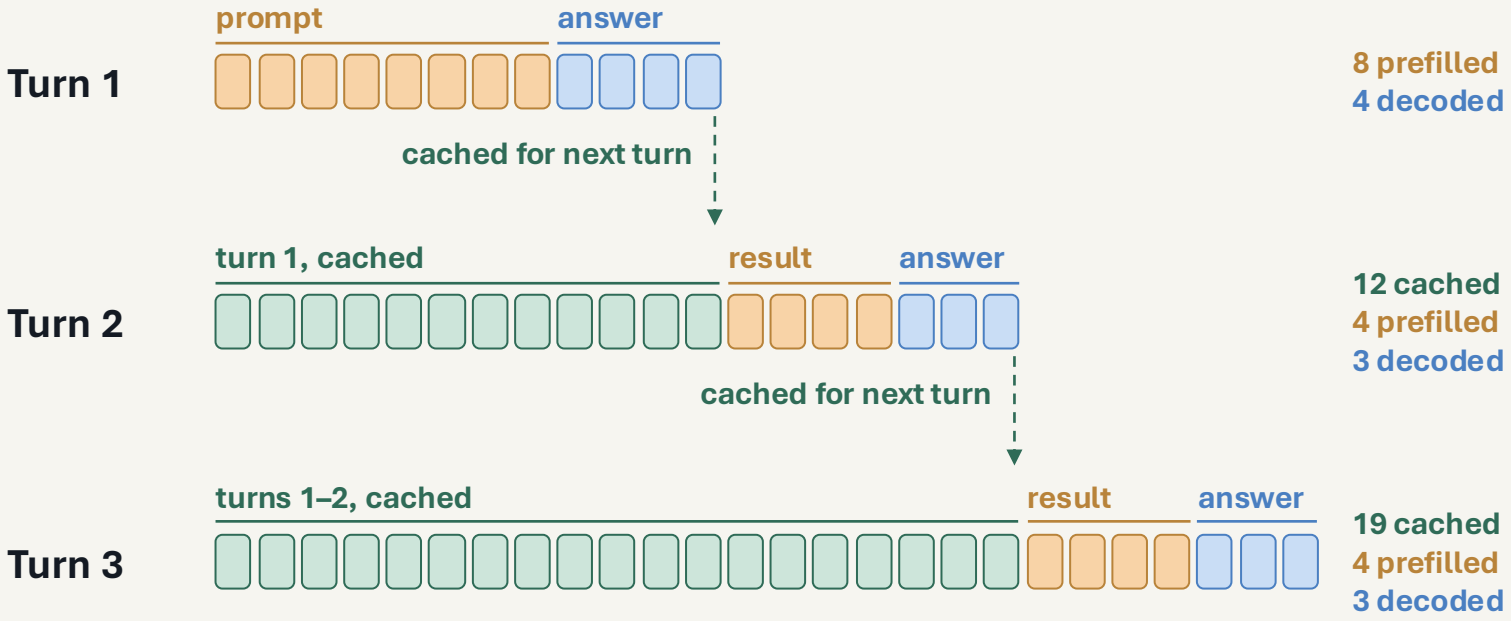
TAKEAWAY Tokens are priced in three categories: input, cached and output.

Token categories in multi-turn inference

Cached read from the KV cache, cheap

New input prefill computes its K, V

Output decode, one token at a time



THE INFERENCE THREAD

- **User** Who designed the stadium by the station?
- **Agent** search("stadium architect")
- **Tool** 3 snippets, top hit doc/203
- **Agent** open("doc/203")
- **Tool** full page of doc/203
- **Agent** “Designed by …”

TAKEAWAY Only new tokens are prefilled each turn.

Example: Q&A over a 100K-token manual on four GPUs



THE DOCUMENT

100K tokens

A product manual, sent in the prompt on every turn.



THE TASK

Q&A, multi-turn

Users ask several short questions about the manual in one chat.



THE HARDWARE

4 GPUs

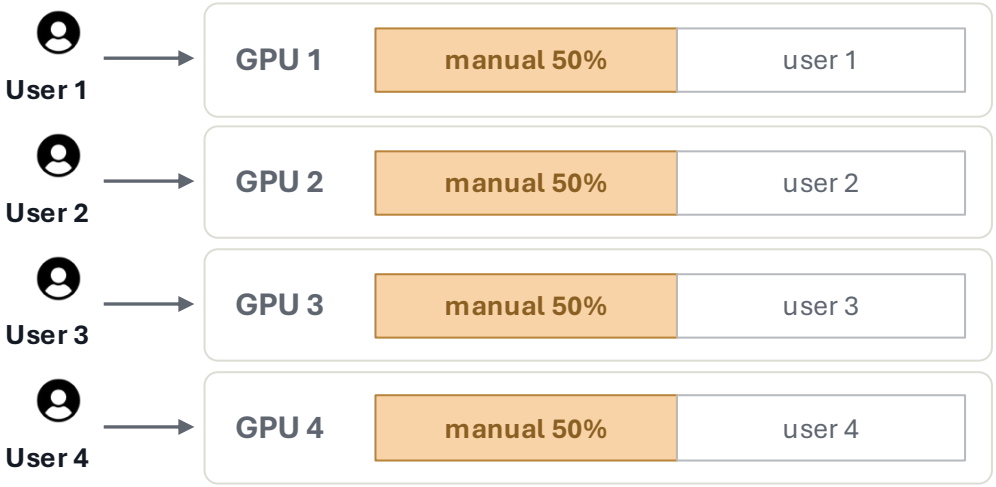
One model served on four GPUs. The choice is how they split the work.

Single GPU inference vs disaggregated inference

STRATEGY A

Single GPU inference

4 independent workers, one per GPU



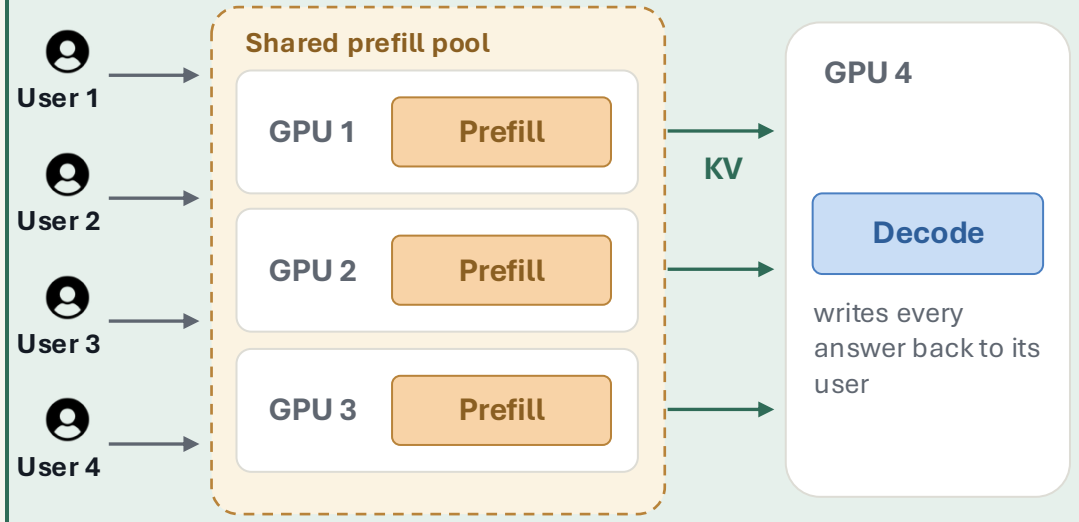
Requests are independent

Max tokens per request: **the free half of one GPU**

STRATEGY B

Disaggregated inference

3 GPUs prefill, 1 GPU decodes every answer



Each request uses a prefill GPU and the decode GPU

Max tokens per request: **more than on the left**

■ Prefill: read the whole prompt, build the KV cache (compute-heavy) ■ Decode: write tokens one at a time (memory-heavy)

When the context fills, the chat wraps and its cache is lost

THE CHAT

SYSTEM product manual, 100K tokens

T1 **USER** What is the warranty period?

AGENT 2 years.

T2 **USER** Does it cover water damage?

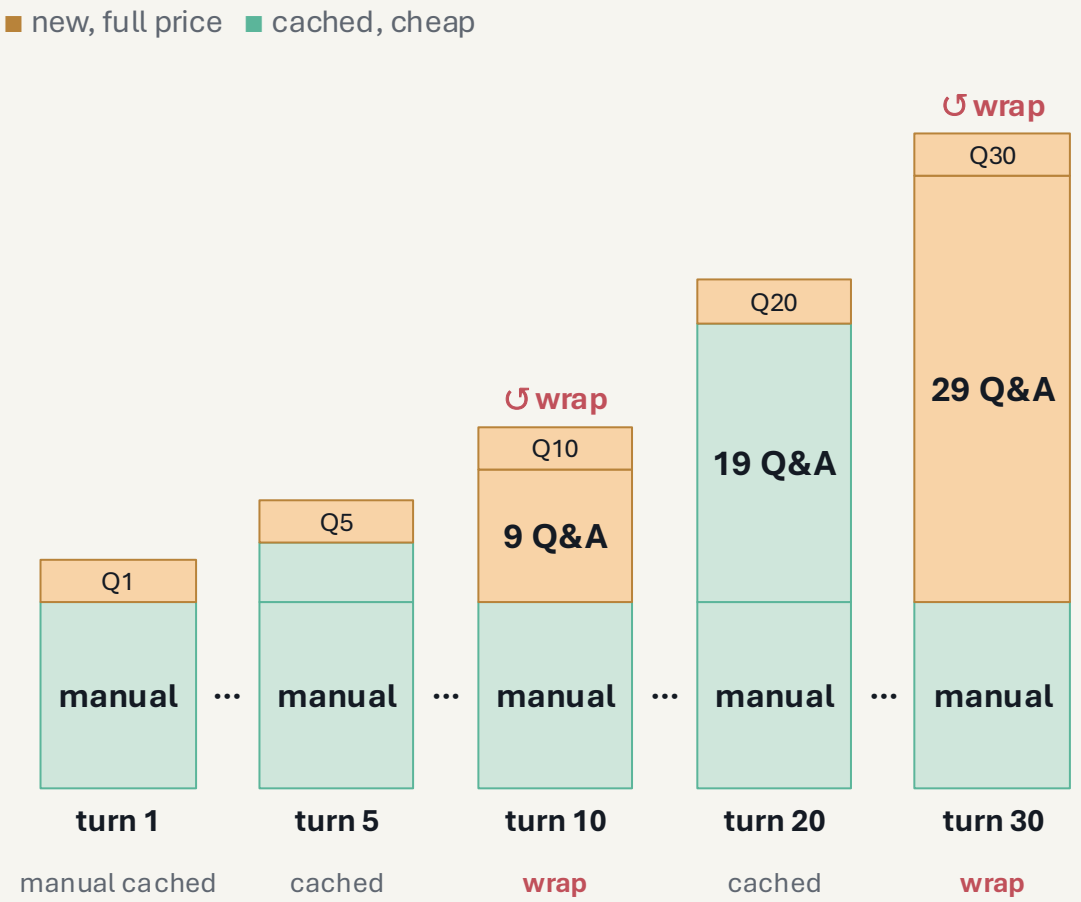
AGENT No, only defects.

↺ context full: wrap

T3 **USER** How do I file a claim?

AGENT Use the online form.

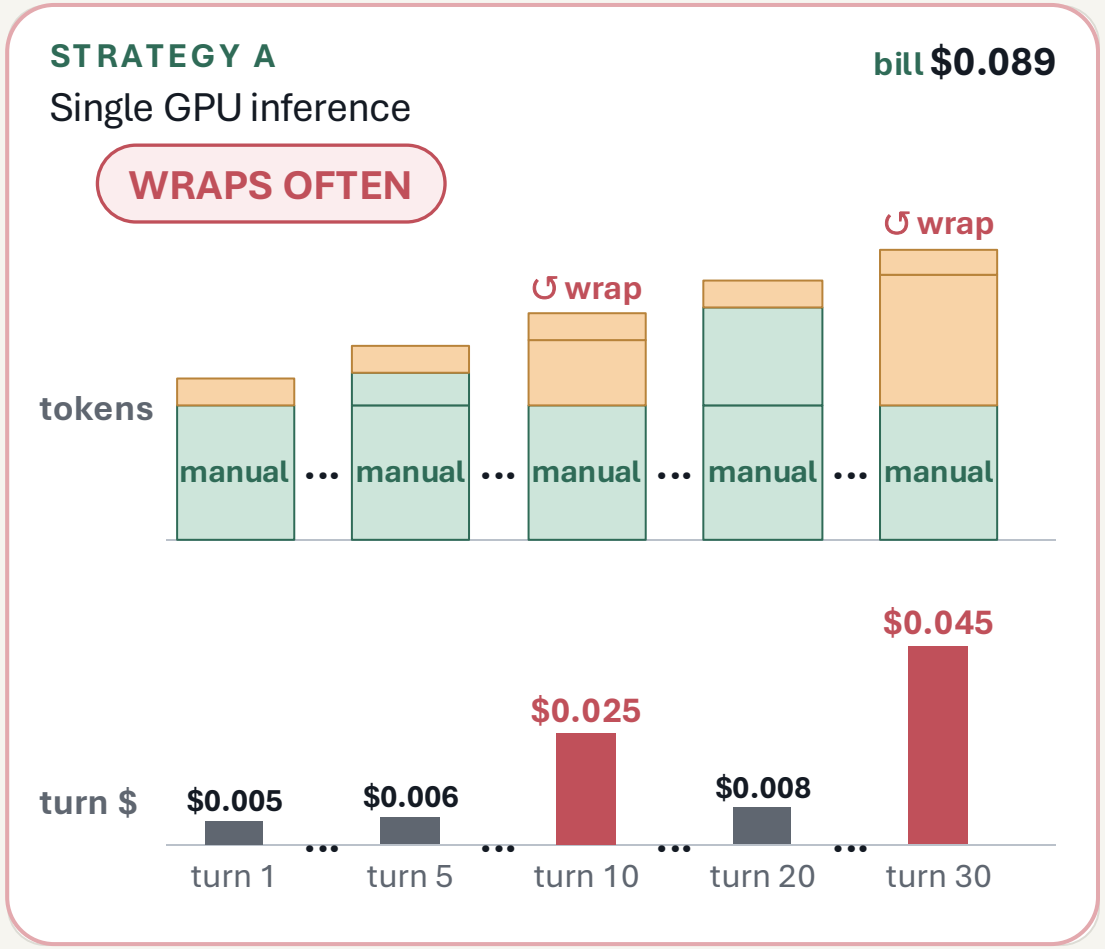
Every turn re-sends the manual and all earlier messages.

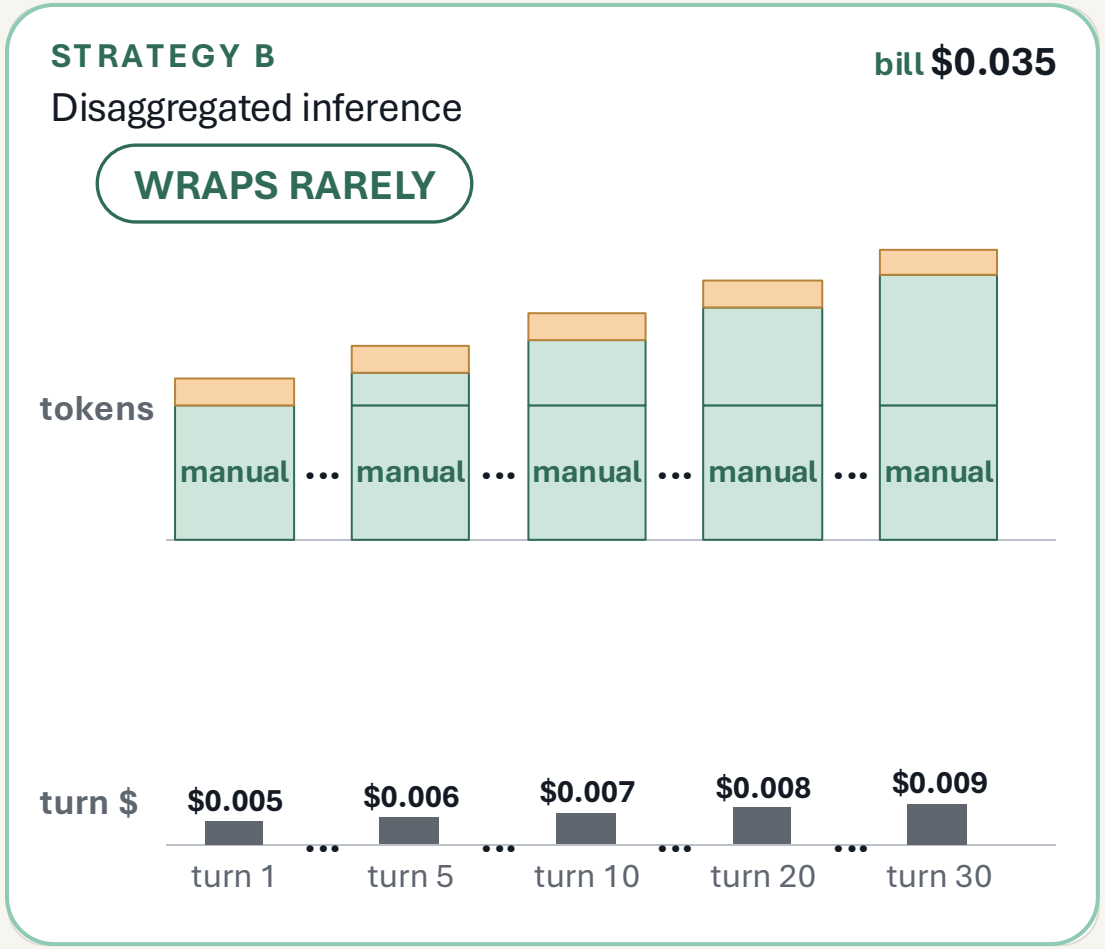


TAKEAWAY Each wrap breaks the cache, so the agent pays full price again.

Fewer context wraps mean more cached tokens and a smaller bill

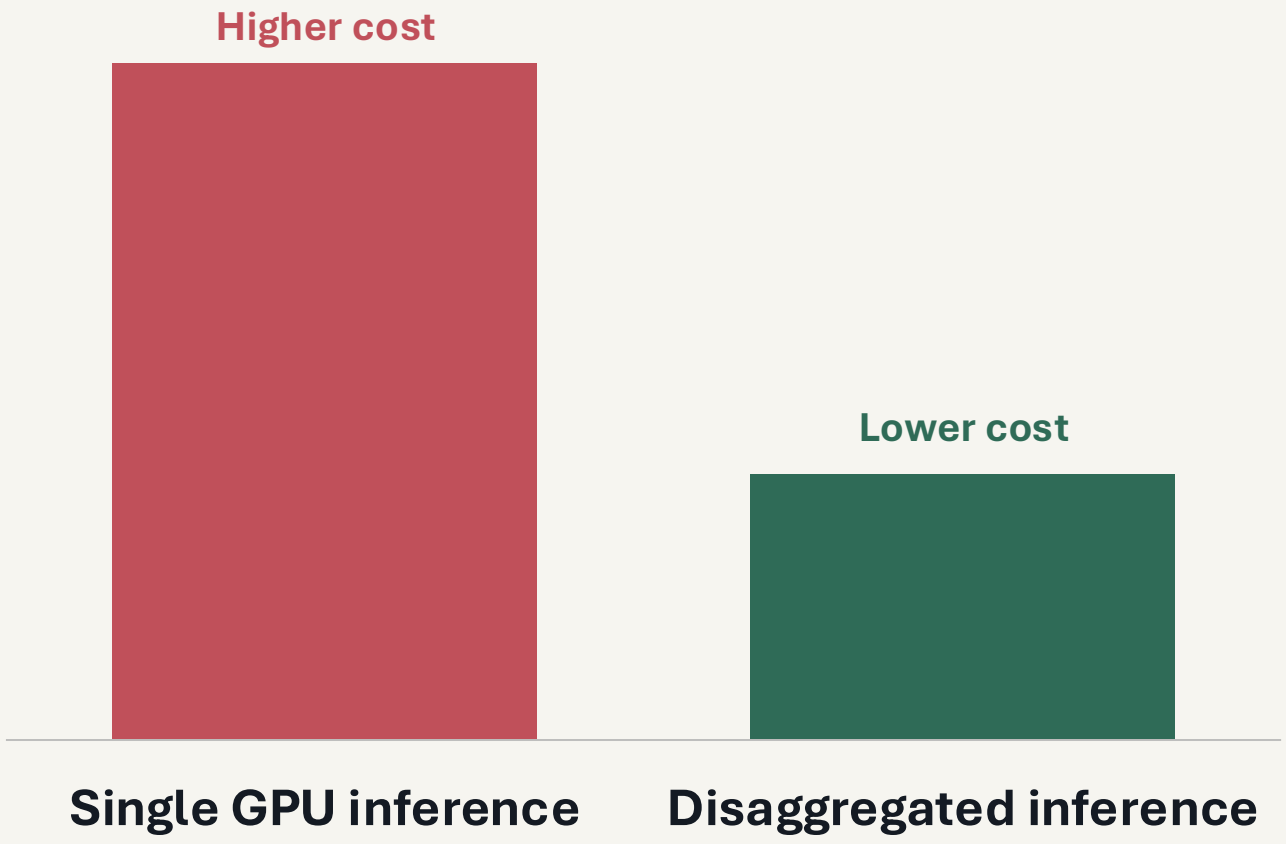
■ new tokens (full price) ■ cached tokens (cheap) ↺ wrap = cache lost





Splitting prefill and decode lowers the inference cost

Inference cost per chat illustrative, same answers



Wrapping often loses the cache

Splitting prefill and decode keeps it warm

Application-aware hardware deployment is needed for cost-efficient inference

Illustrative comparison; bar heights show direction, not measured costs.

Key Takeaways

1

Switch to the best agent as you scale

Pick the agent with the most accuracy per dollar at each budget.

2

Diagnose the agent to find scaling issues

Tracing queries and documents shows why extra turns stall.

3

Plan efficiently down to the hardware

Cache reuse and deployment choices set the real cost per answer.

Thank you

Questions?

avinash@ucf.edu

Appendix

Full detail behind each section, in talk order

tools calls are not free

Official Review of Submission22208 by Reviewer r4ZX

Official Review by Reviewer r4ZX 30 Oct 2025, 19:33 (modified: 12 Nov 2025, 05:08) Everyone Revisions

Summary:

This paper shifts the focus of test-time scaling (TTS) from *token-related scaling* to *tool-call-related scaling*, thereby extending the scope of TTS. It introduces a new problem: how to effectively utilize additional computational budgets allocated for tool calls in tool-augmented agents. To address this, the authors design a *cost-effective agent test-time scaling framework* that scales computation for tool calling with budget-awareness. Using *web search* as a representative task, the proposed scaling method is shown to outperform the base model, several existing trained agents, and some token-based TTS methods.

Soundness: 2: fair
Presentation: 2: fair
Contribution: 3: good

Strengths:

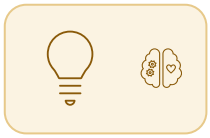
1. The paper presents a novel research problem — *test-time scaling under a tool-call budget* — which fills an important gap in current TTS research and is of high research value.
2. The proposed budget-aware planning and balanced budget usage are thoughtful and practically meaningful design considerations.
3. The problem formulation and motivation are clearly written, especially in the early sections of the paper.

Weaknesses:

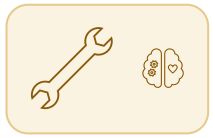
1. The writing quality fluctuates significantly. The first two sections are well-organized and easy to follow, but the later sections, particularly Section 3, lack critical technical details. While the introduction mentions key challenges such as balancing different budget usage across tools and budget-aware exploration, the method section does not clearly explain how these challenges are addressed. Furthermore, several essential details are missing, which hinder reproducibility — for example, how the agent selects tools, how subsequent planning steps utilize previous tool outputs, and how the agent balances between breadth and depth of exploration. The paper would benefit from additional equations and figures to improve clarity. Finally, the early termination mechanism mentioned in line 398 lacks a corresponding description in the method section, and line 472 contains a capitalization issue.
2. The unified metric introduced in Equation (2) lacks generalization. The definition of *economic cost* does not apply uniformly to both local model usage and local tool functions because the models and tools in local are free.
3. The problem setup, which assumes that each tool call has an explicitly defined budget, is unrealistic. In practice, users typically specify a *total* computational budget, and the allocation across tools should be determined by the proposed method. Moreover, the budget definition should jointly consider both *token budget* and *tool-call budget* to align with the broader definition of test-time scaling.
4. The experiments are insufficient. (1) The main experiment should focus on demonstrating how the proposed method improves the *scaling cost–performance curve*, rather than only comparing with base or parallel models. Figure 2's comparison is therefore inadequate. (2) The choice of TTS baselines is limited. The token-based baselines do not include current state-of-the-art methods, and the paper does not explore tool-call-related scaling methods, such as simple BoN or self-refinement on tool calls. (3) Evaluating on only one type of task is insufficient to demonstrate the method's generalization. (4) The ablation studies in Section 5.3 are not well-designed. The ablation of the planning and budget-tracker modules should be conducted within the proposed framework itself, not on another existing method.

The scaffold adds specialized LLM agents and memory

LLM AGENTS same backbone, each with its own prompt and job



Thinker
Splits the question



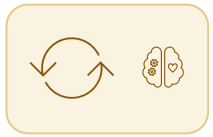
Refiner
Writes the tool calls



Selector
Keeps relevant docs



Summarizer
Condenses long pages



Reflexer
Checks and corrects



Conclusion
Writes the final answer

MEMORY TOOLS where evidence is kept between turns



In-context
Keeps evidence inline in the prompt



Scratchpad
Persistent store for tool outputs and logs

A SCAFFOLD any mix of these agents and memory, wired to `search` · `open` · `find`

Every agent and memory call is metered, so each scaffold has its own cost profile.

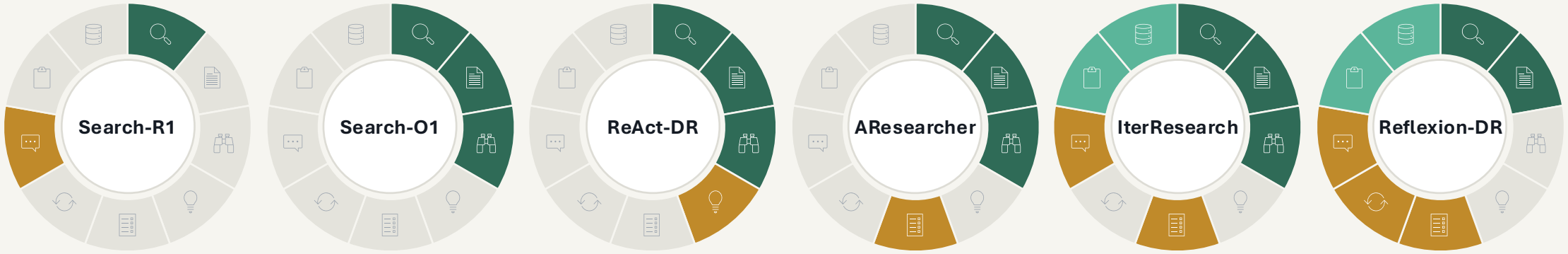
Agents are built from agents: six combinations we tested

TOOLS search · open · find

MEMORY In-context · Scratchpad

LLM AGENTS Thinker · Summarizer · Reflexer · Conclusion

colored = part used gray = not used



COMPOSITION Each agent is the backbone plus a set of smaller agents and memory. **More parts, more metered calls.**

Colored sector = component used. All six run on the same backbone, tools and rate card.

Same task and same prices for every agent we test

BROWSECOMP-PLUS
200/830 multi-constraint questions

FIXED CORPUS
~100K documents, not the live web

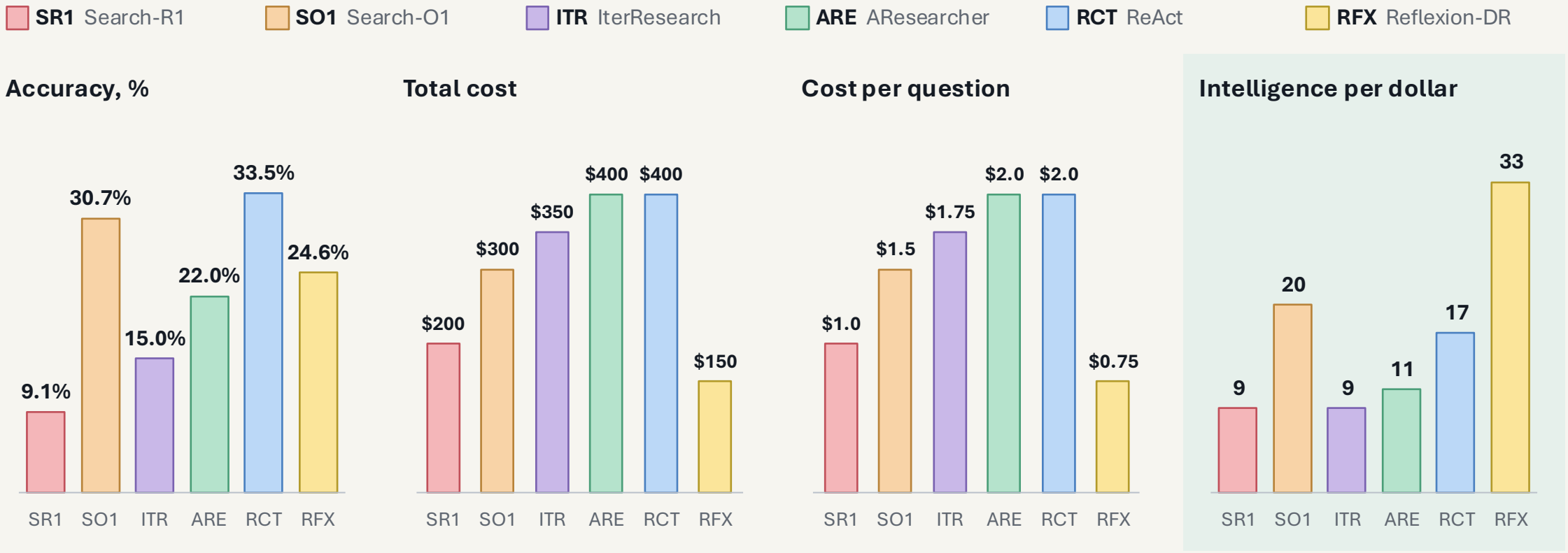
GRADING
1 short answer vs ground truth

CATEGORY	COMPONENT	INPUT	OUTPUT	WHAT IT IS
Browser	`search`	`$0.01/call`	–	SerpAPI query
	`open`	`$0.0067/call`	–	Scrape and clean up the page
	`find`	`$0.0067/call`	–	In-page lookup
Memory	In-context	`$0.05/M`	–	Context always available to the agent
	Scratchpad	`$0.00006/MB`	–	Storage for full trajectory logs
Agent scaffold	Refiner	`$0.05/M`	`$0.24/M`	Turns queries into tool calls
	Selector	`$0.05/M`	`$0.24/M`	Selects relevant documents
	Summarizer	`$0.05/M`	`$0.24/M`	Condenses long content
	Conclusion	`$0.05/M`	`$0.24/M`	Writes the final answer from logs
Backbone	NeMo-N-30B A3B	`$0.05/M`	`$0.24/M`	Model token pricing

USER-SPECIFIC These are our prices. Swap in your own provider's rates and the same metric applies.

BrowseComp-Plus (Chen et al., 2025). /M = per 1M tokens. Model pricing from the Vercel AI Gateway; search from SerpAPI.

ReAct scores highest, but Reflexion-DR buys the most per dollar

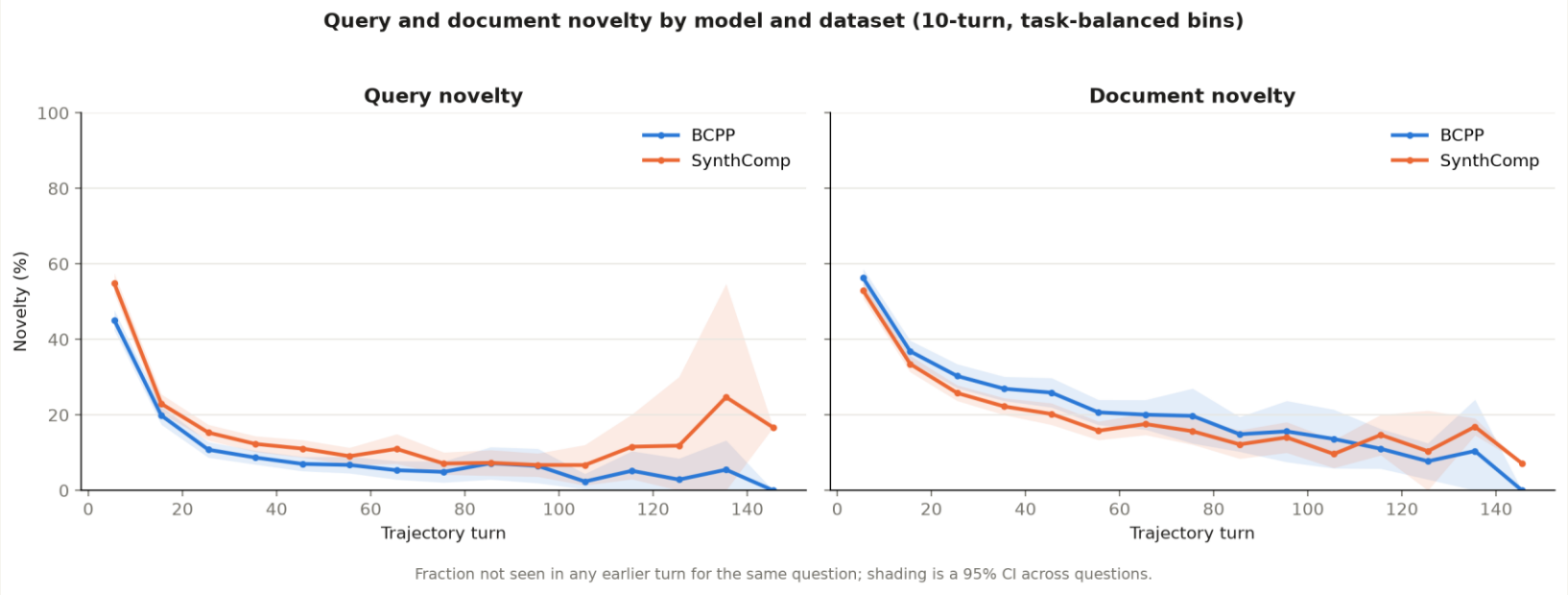


ReAct leads on accuracy, yet Reflexion-DR buys twice its intelligence per dollar.

Illustrative values, matching the scaling and frontier slides; not measured results. Intelligence per dollar = accuracy ÷ cost per question.

After a few turns, new searches stop finding new documents

New documents found per turn one question, illustrative



Budget keeps buying searches, but the corpus it reaches stops growing, **so accuracy flattens.**

Illustrative trajectory; measure as unique documents retrieved per turn in the logs.